

COMPLETE TECHNICAL SPECIFICATIONS DOCUMENT

Architecture | Development Guide | API Integrations | Enhancements

WhatsApp Smart FAQ Bot & Social Media AutoResponder System

ISETAG University — Full Technical Reference v1.1

Part I Architecture & Tech Stack	Part II Database, APIs & Modules	Part III Sprints, Deploy & Security	Part IV Enhancements: Audio & Analytics
---	---	--	--

Client ISETAG University	Document Full Technical TSD v1.1	Prepared by TENEFO S.Y.J. & LEMBOIGNY S.N.	Date July 2026
------------------------------------	---	---	--------------------------

TABLE OF CONTENTS

1. INTRODUCTION & TECHNICAL CONTEXT	5
1.1 Purpose of this document.....	5
1.2 Technical scope	5
2. GLOBAL SYSTEM ARCHITECTURE	6
2.1 Overview — 3-Layer Architecture	6
2.2 Complete message flow — all channels	6
2.3 System component diagram	7
3. RECOMMENDED TECHNOLOGY STACK	8
3.1 Technology decision table	8
3.2 Key npm dependencies	9
4. DATABASE SCHEMA	10
4.1 PostgreSQL — All Tables.....	10
Table: faqs.....	10
Table: conversations.....	10
Table: messages	11
Table: admins	11
Table: broadcasts	11
4.2 Redis Strategy — Cache & Sessions	12
5. API INTEGRATIONS — COMPLETE REFERENCE.....	13
5.1 WhatsApp Cloud API (Meta).....	13
5.1.1 Setup prerequisites	13
5.1.2 Endpoints used	13
5.1.3 Webhook signature verification	13
5.2 Meta Graph API — Facebook & Instagram.....	13
5.2.1 Required OAuth permissions	13
5.2.2 Automatic comment reply flow	14
5.3 OpenAI API — GPT-4o-mini + Whisper.....	14
5.3.1 Two-level NLP strategy.....	14
5.3.2 GPT system prompt for FAQ matching	14
5.3.3 Whisper API — audio transcription	15
6. BACKEND MODULES — DETAILED SPECIFICATIONS.....	16
6.1 Complete project folder structure.....	16
6.2 Module: Session Manager (session.service.js).....	16
6.3 Module: FAQ Engine — Two-pass algorithm.....	17
6.4 Module: Audio Service (audio.service.js).....	17
Integration in whatsapp.js webhook handler	18
6.5 Module: Escalation Manager	18
7. ADMIN DASHBOARD — FRONTEND SPECIFICATIONS	19
7.1 Pages, features and API endpoints	19

7.2 Authentication & security	19
7.3 Analytics page — React component structure	20
7.4 Analytics API endpoints.....	20
7.5 Analytics SQL queries	20
Messages per day (last 30 days)	20
Top 10 most triggered FAQs.....	20
Escalation rate (last 30 days).....	21
Top 20 unmatched questions (knowledge gaps)	21
8. DEVELOPMENT PLAN — SPRINT-BY-SPRINT	22
Sprint 1 — Week 1: Infrastructure & Foundations.....	22
Sprint 2 — Week 2: Core WhatsApp Bot.....	22
Sprint 3 — Week 3: NLP, Audio & Escalation.....	23
Sprint 4 — Week 4: Social Media AutoResponder	23
Sprint 5 — Week 5: Admin Dashboard (Core).....	24
Sprint 6 — Week 6: Analytics Dashboard.....	24
Sprint 7 — Weeks 7–8: Testing, Deployment & Training	25
9. SECURITY REQUIREMENTS	26
9.1 Application security matrix.....	26
9.2 Complete environment variables	27
10. DEPLOYMENT & INFRASTRUCTURE	28
10.1 Infrastructure cost breakdown	28
10.2 Dockerfile — Backend.....	28
10.3 docker-compose.yml — Local development	28
10.4 CI/CD Pipeline — GitHub Actions.....	29
11. ENHANCEMENT 1 — AUDIO MESSAGE PROCESSING.....	30
11.1 Rationale.....	30
11.2 Full audio processing pipeline	30
11.3 Whisper API specifications	31
11.4 Implementation — audio.service.js (complete)	31
11.5 Acceptance criteria — Audio Processing.....	32
12. ENHANCEMENT 2 — BASIC ANALYTICS DASHBOARD	33
12.1 Rationale.....	33
12.2 Complete metrics specification	33
12.3 SQL queries for each metric	34
Messages per day	34
Top 10 triggered FAQs	34
Escalation rate	34
Top 20 knowledge gaps (unmatched messages)	34
Audio vs text message ratio	34
12.4 Analytics acceptance criteria	35
13. COMPLETE ACCEPTANCE CRITERIA	36

14. TECHNICAL RISK MANAGEMENT	37
15. TECHNICAL GLOSSARY	38

PART I

Architecture, Technology Stack & System Design

1. INTRODUCTION & TECHNICAL CONTEXT

1.1 Purpose of this document

This document is the Complete Technical Specifications Document (TSD v1.1) for the WhatsApp FAQ Bot & Social Media AutoResponder project at ISETAG University. It is addressed exclusively to developers, software architects, and technical project managers responsible for building the system.

It covers everything a developer needs to build this system from scratch: architecture, technology choices, database schema, API integrations, code structure, sprint plan, deployment, security, and two enhancement modules (audio processing and analytics). This is the single source of technical truth for the project.

1.2 Technical scope

The system consists of five integrated subsystems:

- An intelligent WhatsApp Bot (NLP + business logic engine)
- A Social Media AutoResponder for Facebook, Instagram, and TikTok
- A web-based Admin Dashboard for the ISETAG management team
- An Audio Processing module (Speech-to-Text via OpenAI Whisper)
- A Basic Analytics Dashboard (monitoring & insights)

INFO

This document is a developer blueprint. Functional requirements (what the system does) are defined in the separate project presentation document. This document focuses exclusively on HOW to build it.

2. GLOBAL SYSTEM ARCHITECTURE

2.1 Overview — 3-Layer Architecture

The system adopts a layered architecture with clear separation between presentation, business logic, and data. This ensures maintainability, scalability, and testability of every component independently.

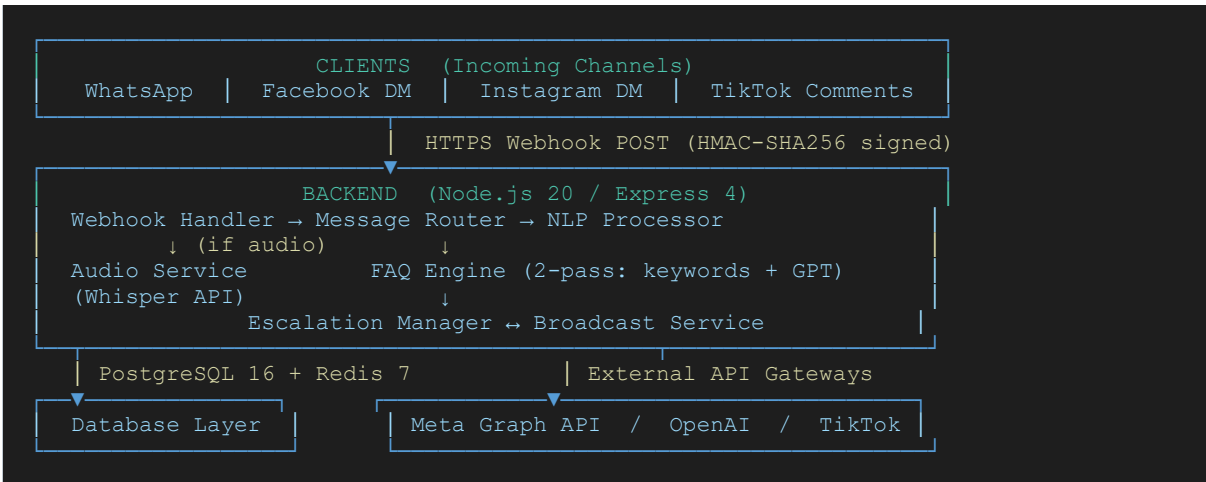
Layer	Components	Technologies
Presentation (Frontend)	Admin Web Dashboard, Analytics Page, Monitoring Interface	React.js 18 + Vite + Tailwind CSS
Business Logic (Backend)	Bot Engine, NLP Processor, Audio Processor, Webhook Handler, Social Media Manager, Broadcast Service	Node.js 20 LTS + Express 4
Data (Backend)	FAQ database, logs, conversations, statistics, sessions	PostgreSQL 16 + Redis 7 (cache)

2.2 Complete message flow — all channels

Every incoming message regardless of channel follows this sequence:

1. User sends a message on WhatsApp, Facebook, Instagram, or TikTok.
2. The platform webhook (Meta Cloud API) forwards the POST request in real time to the backend server.
3. The Webhook Handler authenticates the request (HMAC-SHA256 signature) and routes to the Message Router.
4. The Message Router identifies: channel, message type (text / audio / image / document).
5. If audio: download .ogg file, send to OpenAI Whisper, receive transcript text — then continue as text.
6. The NLP Processor analyzes the text: language detection, intent extraction, spell normalization.
7. The FAQ Engine runs a two-pass search (keyword match, then GPT semantic match if needed).
8. If confidence < 0.5 after both passes: trigger escalation flow with GPT-generated conversation summary.
9. Response formatted for the channel and sent via the corresponding API. Conversation logged to PostgreSQL.

2.3 System component diagram



3. RECOMMENDED TECHNOLOGY STACK

3.1 Technology decision table

Component	Chosen Technology	Alternative	Justification
Main Backend	Node.js 20 LTS + Express 4	Python 3.12 + FastAPI	Excellent WhatsApp libs, native async handling
NLP / AI	OpenAI GPT-4o-mini via API	Dialogflow CX, Rasa NLU	Best EN/FR understanding, low latency, cost-effective
Speech-to-Text	OpenAI Whisper API	Google Speech-to-Text	Same API key, simpler setup, \$0.006/min, accepts .ogg directly
Primary Database	PostgreSQL 16	MySQL 8	Robustness, native JSON, TEXT[] arrays for keywords
Cache & Sessions	Redis 7	Memcached	Real-time bot session management
Admin Dashboard	React 18 + Vite + Tailwind CSS	Vue 3 + Nuxt	Mature ecosystem, recharts for analytics
WhatsApp API	Meta WhatsApp Cloud API (official)	Twilio WhatsApp API	Free up to 1,000 conversations/month
Social Media	Meta Graph API v20+	—	Only official solution for Facebook/Instagram
Hosting	Railway.app or Render.com	DigitalOcean, AWS EC2	Simple, scalable, low cost
CI/CD	GitHub Actions	GitLab CI	Free, native GitHub integration
Monitoring	UptimeRobot + Sentry	Datadog	Free/freemium, sufficient for this scope
File Storage	Cloudinary (Free: 25 GB)	AWS S3	PDFs/images shared via WhatsApp

3.2 Key npm dependencies

Package	Version	Usage
express	^4.18	HTTP server, webhook routing
openai	^4.x	GPT-4o-mini for NLP + Whisper for audio transcription
pg + pg-pool	^8.x	PostgreSQL connection
ioredis	^5.x	Redis client for sessions and cache
langdetect	^1.x	Automatic EN/FR language detection
natural / compromise	^6.x / ^14.x	Lightweight NLP, tokenization, keyword extraction
multer + cloudinary	^1.x / ^2.x	PDF upload and file management
jsonwebtoken + bcrypt	^9.x / ^5.x	Dashboard auth (JWT + password hashing)
joi	^17.x	Incoming payload validation
winston	^3.x	Structured logging
node-cron	^3.x	Scheduled tasks (broadcast sending)
recharts	^2.x	Analytics charts in React dashboard
node-fetch + form-data	^3.x / ^4.x	Audio file download + Whisper API upload

PART II

Database Schema, API Integrations & Backend Modules

4. DATABASE SCHEMA

4.1 PostgreSQL — All Tables

Table: faqs

```
CREATE TABLE faqs (  
  id          SERIAL PRIMARY KEY,  
  category    VARCHAR(100) NOT NULL,      --  
  'admission','fees','programs','dates','contacts'  
  lang        CHAR(2) DEFAULT 'en',      -- 'en' or 'fr'  
  keywords     TEXT[],                    -- array of matching keywords  
  question    TEXT NOT NULL,  
  answer      TEXT NOT NULL,  
  media_url   TEXT,                      -- optional PDF or image CDN link  
  is_active   BOOLEAN DEFAULT TRUE,  
  created_at  TIMESTAMP DEFAULT NOW(),  
  updated_at  TIMESTAMP DEFAULT NOW()  
);
```

Table: conversations

```
CREATE TABLE conversations (  
  id          SERIAL PRIMARY KEY,  
  channel     VARCHAR(20) NOT NULL,      --  
  'whatsapp','facebook','instagram','tiktok'  
  user_phone  VARCHAR(30),               -- anonymized SHA-256 hash  
  session_id  UUID DEFAULT gen_random_uuid(),  
  status      VARCHAR(20) DEFAULT 'bot', -- 'bot' | 'escalated' | 'closed'  
  lang_detected CHAR(2),                 -- 'en' | 'fr'  
  summary     TEXT,                      -- AI-generated summary before  
  escalation  
  created_at  TIMESTAMP DEFAULT NOW(),  
  closed_at   TIMESTAMP  
);
```

Table: messages

```
CREATE TABLE messages (  
  id SERIAL PRIMARY KEY,  
  conversation_id INTEGER REFERENCES conversations(id),  
  direction VARCHAR(3), -- 'in' | 'out'  
  content TEXT, -- text content (or transcript for  
audio)  
  msg_type VARCHAR(20), -- 'text'|'audio'|'document'|'image'  
  faq_matched_id INTEGER REFERENCES faqs(id), -- null if no match found  
  confidence FLOAT, -- similarity score (0-1)  
  sent_at TIMESTAMP DEFAULT NOW()  
);
```

Table: admins

```
CREATE TABLE admins (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(80) UNIQUE NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  password TEXT NOT NULL, -- bcrypt hash  
  role VARCHAR(20) DEFAULT 'editor', -- 'admin' | 'editor'  
  last_login TIMESTAMP,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Table: broadcasts

```
CREATE TABLE broadcasts (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(200),  
  message TEXT NOT NULL,  
  media_url TEXT,  
  channels TEXT[], -- ['whatsapp','facebook']  
  status VARCHAR(20) DEFAULT 'draft', -- 'draft'|'scheduled'|'sent'  
  scheduled_at TIMESTAMP,  
  sent_at TIMESTAMP,  
  created_by INTEGER REFERENCES admins(id),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

4.2 Redis Strategy — Cache & Sessions

Redis Key (pattern)	TTL	Content
session:{phone_hash}	30 min	Active conversation state: menu step, language, message history, escalation flag
faq:cache:{lang}	1 hour	All active FAQs in memory — avoids repeated SELECT queries
ratelimit:{phone_hash}	1 min	Anti-spam counter: max 10 messages per minute per user
escalation:{session_id}	24h	Flag indicating a human agent has taken over
audio:processing:{msg_id}	5 min	Temporary flag while Whisper API processes a voice note

5. API INTEGRATIONS — COMPLETE REFERENCE

5.1 WhatsApp Cloud API (Meta)

5.1.1 Setup prerequisites

10. Create a Meta Business Suite account at business.facebook.com
11. Create a Business-type Meta app in the Meta Developer Portal
12. Enable the WhatsApp Business Platform product in the app
13. Obtain a dedicated phone number (real number or sandbox test number)
14. Generate a permanent access token (System User Token recommended)
15. Configure the webhook URL on the server (HTTPS endpoint mandatory — use ngrok in development)
16. Verify the webhook with the Meta challenge (hub.verify_token parameter)

5.1.2 Endpoints used

Method	Endpoint	Usage	Key Parameters
POST	/v20.0/{phone_id}/messages	Send text/interactive message	to, type, text/interactive/document
GET	/v20.0/{media_id}	Get download URL for received media	Authorization: Bearer {token}
GET	{media_download_url}	Download the actual audio/image file	Authorization: Bearer {token}
POST	Incoming Webhook	Receive all messages	HMAC-SHA256 signed payload

5.1.3 Webhook signature verification

```
const crypto = require('crypto');
function verifyWebhook(req, res, buf) {
  const sig = req.headers['x-hub-signature-256'];
  const expected = 'sha256=' + crypto
    .createHmac('sha256', process.env.WHATSAPP_APP_SECRET)
    .update(buf).digest('hex');
  if (sig !== expected) throw new Error('Invalid signature - reject request');
}
```

5.2 Meta Graph API — Facebook & Instagram

5.2.1 Required OAuth permissions

- pages_messaging — Read and send Facebook DMs

- `instagram_basic` — Basic Instagram account access
- `instagram_manage_messages` — Read and reply to Instagram DMs
- `pages_read_engagement` — Read comments on posts
- `pages_manage_posts` — Reply to comments
- `public_profile` — Basic profile information

IMPORTANT

All permissions require Meta App Review before production. In sandbox/development mode only 5 test accounts can interact. Plan 2–4 weeks for App Review in your project timeline.

5.2.2 Automatic comment reply flow

17. Webhook receives a feed event (new comment posted on a page or reel)
18. Extract: `page_id`, `comment_id`, `from.id` (author), `message` (comment text)
19. Classify comment via NLP (admission / fees / programs / other)
20. If classification score > 0.65: reply via `POST /v20.0/{comment_id}/comments`
21. If score <= 0.65: notify Community Manager via internal WhatsApp notification

5.3 OpenAI API — GPT-4o-mini + Whisper

5.3.1 Two-level NLP strategy

Level	Method	When to use
Level 1 (fast)	Keyword intersection + cosine similarity (local, no API call)	Short messages with well-defined keywords — covers ~85% of cases
Level 2 (GPT)	GPT-4o-mini API call with context of 20 candidate FAQs	Ambiguous messages, complex phrasing, out-of-scope questions

5.3.2 GPT system prompt for FAQ matching

```
const SYSTEM_PROMPT = `
You are the assistant of ISETAG University.
Analyze the student or parent question and return ONLY a JSON object:
{
  intent: string,          // admission|fees|programs|dates|contacts|other
  lang: string,            // en|fr
  faq_id: integer|null,    // ID from the provided FAQ list, null if no match
  confidence: float,       // 0.0 to 1.0
  needs_human: boolean     // true if human agent required
}
Respond ONLY with valid JSON. No text outside the JSON object.
`;
```

5.3.3 Whisper API — audio transcription

Criterion	Detail
Endpoint	POST https://api.openai.com/v1/audio/transcriptions
Model	whisper-1
Supported formats	.ogg, .mp3, .mp4, .wav, .webm (WhatsApp .ogg files accepted directly)
Max file size	25 MB per request (typical WhatsApp voice note: < 2 MB)
Pricing	\$0.006 per minute — a 30-second note costs \$0.003
Latency	1–3 seconds typical for a standard voice note
Language	Auto-detected — handles French, English, and mixed-language messages

6. BACKEND MODULES — DETAILED SPECIFICATIONS

6.1 Complete project folder structure

```
isetag-bot/
├─ src/
│  ├─ config/
│  │  ├─ db.js           # PostgreSQL connection pool
│  │  ├─ redis.js        # Redis client
│  │  └─ env.js          # Environment variable validation (joi)
│  ├─ webhooks/
│  │  ├─ whatsapp.js     # WhatsApp webhook handler (text + audio branch)
│  │  ├─ facebook.js     # Facebook/Instagram webhook handler
│  │  └─ tiktok.js       # TikTok comment handler
│  ├─ services/
│  │  ├─ nlp.service.js  # Language detection + intent extraction
│  │  ├─ faq.service.js  # FAQ two-pass search engine
│  │  ├─ audio.service.js # Whisper transcription (NEW)
│  │  ├─ session.service.js # Redis session CRUD
│  │  ├─ escalation.service.js # Human handover logic + summary generation
│  │  └─ broadcast.service.js # Group message sending
│  ├─ api/
│  │  ├─ faqs.routes.js  # FAQ CRUD (dashboard)
│  │  ├─ stats.routes.js # Analytics endpoints (NEW)
│  │  ├─ auth.routes.js  # Admin login (JWT)
│  │  ├─ media.routes.js # File upload
│  │  └─ broadcast.routes.js # Broadcast send/schedule
│  └─ utils/
│     ├─ logger.js       # Winston structured logger
│     ├─ crypto.js       # Phone number SHA-256 anonymization
│     └─ messageFormatter.js # Channel-specific response formatting
├─ migrations/           # Numbered SQL migration scripts
├─ tests/
│  ├─ unit/              # Jest unit tests per service
│  └─ integration/       # End-to-end flow tests
├─ .env.example
├─ docker-compose.yml
└─ Dockerfile
```

6.2 Module: Session Manager (session.service.js)

Session Field	Description
step	Current menu step: welcome main_menu admission fees programs ...
lang	Detected language: 'en' or 'fr'

Session Field	Description
history	Array of last 5 messages (used as GPT context window for better matching)
is_escalated	Boolean — if true, all messages bypass the bot and go to human agent
agent_notified	Boolean — whether the admin escalation notification has already been sent
last_audio_msg_id	Stores media_id of the most recent audio message (for deduplication)
created_at	Session start timestamp

6.3 Module: FAQ Engine — Two-pass algorithm

22. PASS 1 (Fast, no API call): Intersect message keywords with FAQ keywords from Redis cache. Score = common keywords / total FAQ keywords. If score > 0.7, return answer immediately.
23. PASS 2 (Semantic, GPT): Build a prompt with the 20 most likely FAQs (pre-filtered by detected category) and call GPT-4o-mini. Receive faq_id and confidence score.
24. If confidence < 0.5 after both passes: trigger the Escalation Manager.

6.4 Module: Audio Service (audio.service.js)

```
const fs = require('fs');
const fetch = require('node-fetch');
const OpenAI = require('openai');
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

async function transcribeAudio(mediaId, accessToken) {
  // Step 1: Get the download URL from Meta Media API
  const metaRes = await fetch(
    `https://graph.facebook.com/v20.0/${mediaId}`,
    { headers: { Authorization: `Bearer ${accessToken}` } }
  );
  const { url } = await metaRes.json();

  // Step 2: Download the .ogg audio file
  const audioRes = await fetch(url, {
    headers: { Authorization: `Bearer ${accessToken}` }
  });
  const buffer = await audioRes.buffer();
  const tempPath = `/tmp/audio_${Date.now()}.ogg`;
  fs.writeFileSync(tempPath, buffer);

  // Step 3: Transcribe with OpenAI Whisper
  const transcription = await openai.audio.transcriptions.create({
    file: fs.createReadStream(tempPath),
    model: 'whisper-1',
  });
}
```

```
});

fs.unlinkSync(tempPath); // Clean up temp file
return transcription.text; // Plain text -> feed into NLP pipeline
}
```

Integration in whatsapp.js webhook handler

```
async function handleIncomingMessage(message) {
  let textToProcess = '';

  if (message.type === 'text') {
    textToProcess = message.text.body;
  } else if (message.type === 'audio') {
    textToProcess = await transcribeAudio(
      message.audio.id, process.env.WHATSAPP_TOKEN
    );
    await logMessage(conversationId, 'in', textToProcess, 'audio');
  } else {
    return sendUnsupportedTypeReply(message.from);
  }
  return await nlpService.process(textToProcess, sessionId); // unchanged
}
```

6.5 Module: Escalation Manager

Escalation to a human agent is automatically triggered when:

- 3 consecutive messages return no FAQ match (confidence < 0.5)
- User explicitly requests a human: 'agent', 'human', 'person', 'real person', 'aide personnelle'...
- GPT returns needs_human: true in the JSON response

Escalation sequence:

25. Call GPT: 'Summarize in 3 lines this conversation between a student and the bot' — store result in conversations.summary
26. Send user: 'An advisor will get back to you shortly. Here is a summary of your inquiry: {summary}'
27. Send WhatsApp notification to admin team number with: user reference + summary + timestamp
28. Set is_escalated = true in Redis session; set conversations.status = 'escalated' in PostgreSQL
29. All subsequent messages from that user are logged but no longer trigger bot responses

PART III

Dashboard, Sprint Plan, Security & Deployment

7. ADMIN DASHBOARD — FRONTEND SPECIFICATIONS

7.1 Pages, features and API endpoints

Page	Features	API Endpoints
Login	Email + password form, JWT stored in httpOnly cookie, redirect on success	POST /api/auth/login
Main Dashboard	KPI cards + weekly chart (messages/day, escalation %, top FAQs)	GET /api/stats/overview
FAQ Management	Full CRUD table with category/language filters, edit modal, active toggle	GET/POST/PUT/DELETE /api/faqs
Conversations	Escalated conversations list, filters by date/status, full session detail view	GET /api/conversations
Analytics	6 interactive charts: messages/day, top FAQs, escalation donut, channel pie, unmatched questions table	GET /api/stats/*
Broadcasts	Compose, select channels (WhatsApp/Facebook/Instagram), schedule or send immediately	POST /api/broadcasts
Media & Files	Upload PDFs, view CDN URL, toggle visibility, delete files	POST/DELETE /api/media
Settings	Change passwords, manage admin accounts, configure bot thresholds, view API usage	PUT /api/admins/{id}

7.2 Authentication & security

- JWT with 8h expiry, stored in httpOnly cookie — not accessible from JavaScript
- Refresh token stored in database — server-side invalidation possible at any time
- Login rate limiting: max 5 attempts / 15 min per IP (express-rate-limit middleware)
- All /api/* endpoints protected by verifyJWT middleware
- Roles: admin (full access) / editor (FAQ + broadcasts only)

7.3 Analytics page — React component structure

```
src/pages/Analytics.jsx
├─ components/
│   └─ KPICards.jsx           // 4 summary cards: messages, convos, escalations, avg
confidence
│   └─ MessagesPerDayChart.jsx // Bar chart — last 30 days (recharts BarChart)
│   └─ TopFAQsChart.jsx       // Horizontal bar — top 10 triggered FAQs
│   └─ EscalationDonut.jsx    // Donut — bot-handled vs human-escalated
│   └─ ChannelPieChart.jsx    // Pie — WhatsApp vs Facebook vs Instagram
│   └─ UnmatchedTable.jsx     // Table of top 20 unanswered questions (last 7 days)
├─ hooks/
│   └─ useAnalytics.js        // Fetches all 6 stat endpoints in parallel
(Promise.all)
```

7.4 Analytics API endpoints

Method	Endpoint	Response
GET	/api/stats/overview	{ total_messages, total_conversations, escalation_count, avg_confidence } this week
GET	/api/stats/messages-per-day?days=30	Array of { day: '2025-07-01', count: 42 }
GET	/api/stats/top-faqs?limit=10	Array of { faq_id, question, trigger_count } ordered by frequency
GET	/api/stats/escalation-rate	{ bot_handled, escalated, escalation_pct } for last 30 days
GET	/api/stats/channels	Array of { channel, count } split by platform
GET	/api/stats/unmatched?days=7	Top 20 unmatched questions (faq_matched_id IS NULL) in last N days

7.5 Analytics SQL queries

Messages per day (last 30 days)

```
SELECT DATE(sent_at) as day, COUNT(*) as count
FROM messages WHERE direction = 'in'
AND sent_at >= NOW() - INTERVAL '30 days'
GROUP BY DATE(sent_at) ORDER BY day ASC;
```

Top 10 most triggered FAQs

```
SELECT f.question, f.category, COUNT(m.id) as trigger_count
FROM messages m JOIN faqs f ON m.faq_matched_id = f.id
WHERE m.faq_matched_id IS NOT NULL
GROUP BY f.id, f.question, f.category
ORDER BY trigger_count DESC LIMIT 10;
```

Escalation rate (last 30 days)

```
SELECT
  COUNT(*) FILTER (WHERE status = 'bot') AS bot_handled,
  COUNT(*) FILTER (WHERE status = 'escalated') AS escalated,
  ROUND(COUNT(*) FILTER (WHERE status='escalated')*100.0/COUNT(*),1) AS escalation_pct
FROM conversations WHERE created_at >= NOW() - INTERVAL '30 days';
```

Top 20 unmatched questions (knowledge gaps)

```
SELECT content, COUNT(*) as frequency
FROM messages
WHERE faq_matched_id IS NULL AND direction = 'in'
  AND sent_at >= NOW() - INTERVAL '7 days'
GROUP BY content ORDER BY frequency DESC LIMIT 20;
```

8. DEVELOPMENT PLAN — SPRINT-BY-SPRINT

The project is broken into 7 sprints. Each sprint has numbered tasks with a measurable deliverable. Sprints 1–6 cover the core system; Sprint 7 covers the two enhancements.

Sprint 1 — Week 1: Infrastructure & Foundations

#	Task	Deliverable
1	Create GitHub repository (branching: main / develop / feature/*)	Repo initialized + README + .gitignore
2	Set up local dev environment: Node.js, PostgreSQL, Redis via Docker Compose	docker-compose.yml running all services
3	Create Meta Developer account, WhatsApp Business app, generate System User Token	Tokens documented in .env.example
4	Initialize Express project: folder structure, ESLint, Prettier, Winston logger	Server starting cleanly on port 3000
5	Write and run all SQL migration scripts (all 5 tables)	All tables present and verified in psql
6	Configure WhatsApp webhook: ngrok HTTPS tunnel + Meta challenge verification	Meta dashboard shows webhook as verified

Sprint 2 — Week 2: Core WhatsApp Bot

#	Task	Deliverable
1	Parse all incoming WhatsApp message types (text, audio, document, image)	Structured log entry for every message type
2	Build session.service.js: create, read, update, expire sessions in Redis	Passing unit tests for all 4 session operations
3	Build interactive menu (Meta buttons API): main menu EN/FR + 5 category submenus	Menu navigable end-to-end on test phone
4	Implement language detection (langdetect) with automatic EN/FR session update	Bot replies in correct language on 20 test messages
5	Build FAQ Engine Pass 1: keyword intersection + Redis cache	Correct answer returned on 10 standard test questions
6	Integrate PDF sharing via WhatsApp (Cloudinary URL + document message type)	PDF from Cloudinary received on test device

Sprint 3 — Week 3: NLP, Audio & Escalation

#	Task	Deliverable
1	Integrate GPT-4o-mini: FAQ Engine Pass 2 (semantic matching)	Accuracy > 80% on 50-question annotated test set
2	Implement spell normalization before keyword matching (Pass 1 improvement)	10 messages with typos returning correct answers
3	Build audio.service.js: Meta media download + Whisper transcription	Voice note transcribed and correctly answered on test phone
4	Integrate audio branch in whatsapp.js webhook handler	msg_type='audio' stored correctly in messages table
5	Build Escalation Manager: 3 triggers + GPT summary + admin WhatsApp notification	Escalation triggered and admin receives notification with summary
6	Implement Redis rate limiting per phone number (10 msg/min anti-spam)	11th message within 1 minute returns rate-limit reply

Sprint 4 — Week 4: Social Media AutoResponder

#	Task	Deliverable
1	Configure Meta Graph API webhook for Facebook & Instagram DMs	DM receipt logged server-side with correct channel tag
2	Implement auto-replies to DMs: welcome + FAQ + WhatsApp redirect	Test Facebook DM receiving automated response < 5s
3	Configure webhook for Facebook/Instagram post comments	Comments received, logged, and channel identified
4	Implement comment classification (NLP) and contextual auto-reply	Test comment receiving correctly classified response
5	Build TikTok connector (TikTok for Developers API or scraping fallback)	Repetitive comments detected and auto-reply posted
6	Cross-platform consistency test: same question on 3 channels	All 3 channels return identical core answer content

Sprint 5 — Week 5: Admin Dashboard (Core)

#	Task	Deliverable
1	Build all REST API endpoints (CRUD FAQs, auth, broadcasts, media upload)	Full Postman collection with all endpoints tested
2	Initialize React + Vite + Tailwind CSS + React Router app skeleton	App running on localhost:5173 with route structure
3	Login page: form + API call + JWT cookie + protected route redirect	Login flow working end-to-end
4	FAQ Management page: filterable table + edit modal + active/inactive toggle	Staff can add, edit and deactivate a FAQ from the browser
5	Broadcasts page: compose + channel selection + schedule picker + send	Broadcast delivered to test WhatsApp number
6	Media & Files page: drag-drop upload + Cloudinary integration + file list	PDF uploaded, CDN URL visible, shareable in bot

Sprint 6 — Week 6: Analytics Dashboard

#	Task	Deliverable
1	Write 4 PostgreSQL analytics queries + build stats.routes.js (6 endpoints)	All endpoints return correct data vs direct psql queries
2	Build KPICards.jsx component (4 summary metric cards)	Cards displaying real data from seeded test conversations
3	Build MessagesPerDayChart.jsx + TopFAQsChart.jsx (bar charts with recharts)	Charts rendering correctly for 7, 14, and 30-day ranges
4	Build EscalationDonut.jsx + ChannelPieChart.jsx (donut and pie charts)	Charts update immediately after new test conversations
5	Build UnmatchedTable.jsx showing top 20 unanswered questions	5 test unanswerable questions appear in table within 1 min
6	Build useAnalytics.js hook — parallel fetch of all 6 endpoints	Analytics page fully loaded in < 3 seconds

Sprint 7 — Weeks 7–8: Testing, Deployment & Training

#	Task	Deliverable
1	Jest unit tests: all services (NLP, FAQ, Session, Audio, Escalation) — coverage > 70%	Jest HTML coverage report
2	Integration tests: full message flows (text bot, audio bot, escalation, social media)	All integration scenarios passing
3	Load testing with k6: simulate 500 concurrent users for 2 minutes	P95 response time < 4s under full load
4	Deploy to Railway.app: backend + PostgreSQL + Redis + all env vars secured	Production URL live and accessible
5	Deploy frontend to Vercel: connect to production API URL, configure env	Dashboard accessible at custom URL
6	Set up CI/CD GitHub Actions: lint + Jest + deploy on merge to main	Pipeline runs successfully on a test PR
7	Set up UptimeRobot monitoring: 5-min checks + email alerts	Monitor active, first alert successfully received
8	Staff training session (Community Manager + Admissions Office)	Training delivered, user guide document handed over

9. SECURITY REQUIREMENTS

9.1 Application security matrix

Threat	Technical Countermeasure
Forged webhook request	HMAC-SHA256 signature verification on every incoming Meta request
SQL injection	Parameterized queries exclusively — no string concatenation in SQL
Login brute force	express-rate-limit: 5 attempts / 15min per IP + account lockout
Dashboard XSS	Helmet.js + Content Security Policy + React native escaping
Exposed API tokens	Environment variables only — never hardcoded, never committed to Git
User spam	Redis rate limiter: max 10 messages per minute per phone number
Personal data exposure	Phone numbers hashed with SHA-256 before any database storage
Session hijacking	httpOnly cookies for JWT, refresh tokens in DB, logout invalidates all
Audio file abuse	Temp files deleted immediately after Whisper API call (fs.unlinkSync)

9.2 Complete environment variables

```
# — WhatsApp & Meta —————
WHATSAPP_TOKEN=EAAxxxxxxxxx
WHATSAPP_PHONE_NUMBER_ID=1234567890
WHATSAPP_APP_SECRET=abc123...      # for webhook HMAC verification
WHATSAPP_VERIFY_TOKEN=my_verify_token # for webhook setup challenge
FACEBOOK_PAGE_ACCESS_TOKEN=EAAxxxxxxxxx
INSTAGRAM_PAGE_ID=123456

# — OpenAI (GPT + Whisper) —————
OPENAI_API_KEY=sk-xxxxxxxxx        # used for both GPT-4o-mini and Whisper

# — Database —————
DATABASE_URL=postgresql://user:pass@host:5432/isetag_bot
REDIS_URL=redis://localhost:6379

# — Dashboard Auth —————
JWT_SECRET=a_very_long_and_random_secret_string
JWT_EXPIRES_IN=8h

# — Cloudinary (file storage) —————
CLOUDINARY_CLOUD_NAME=isetag
CLOUDINARY_API_KEY=xxxxxxxxx
CLOUDINARY_API_SECRET=xxxxxxxxx

# — App —————
PORT=3000
NODE_ENV=production
ADMIN_WHATSAPP_NUMBER=+237600000000 # number to receive escalation alerts
```

10. DEPLOYMENT & INFRASTRUCTURE

10.1 Infrastructure cost breakdown

Service	Platform	Cost/month
Node.js Backend	Railway.app (Starter)	~\$5 USD/month
PostgreSQL 16	Railway.app (Plugin)	Included in above
Redis 7	Railway.app (Plugin)	~\$3 USD/month
Admin Dashboard + Analytics	Vercel (Free tier)	\$0
File Storage (PDFs, images)	Cloudinary (Free: 25 GB)	\$0
Uptime Monitoring	UptimeRobot (Free)	\$0
Error Tracking	Sentry (Free tier: 5k errors/mo)	\$0
OpenAI Whisper (audio)	Per-minute usage	~\$2-5 USD/month (at ~500 voice notes)
TOTAL		~\$10-13 USD/month

10.2 Dockerfile — Backend

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY src/ ./src/
EXPOSE 3000
CMD ["node", "src/index.js"]
```

10.3 docker-compose.yml — Local development

```
version: '3.9'
services:
  backend:
    build: .
    ports: ['3000:3000']
    env_file: .env
    depends_on: [postgres, redis]
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: isetag_bot
      POSTGRES_USER: dev
      POSTGRES_PASSWORD: devpass
    ports: ['5432:5432']
```

```
volumes: ['pgdata:/var/lib/postgresql/data']
redis:
  image: redis:7-alpine
  ports: ['6379:6379']
volumes:
  pgdata:
```

10.4 CI/CD Pipeline — GitHub Actions

```
name: Test and Deploy
on:
  push:
    branches: [main]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage --ci
      - uses: bervProject/railway-deploy@main
        with:
          railway_token: ${ secrets.RAILWAY_TOKEN }
          service: isetag-bot-backend
```

PART IV

System Enhancements: Audio Processing & Analytics

INFO

Part IV formalizes two additions approved as part of TSD v1.1. Both enhancements integrate into the existing architecture without rebuilding any core module. Their implementation is already included in the sprint plan (Sprints 3 and 6 respectively).

11. ENHANCEMENT 1 — AUDIO MESSAGE PROCESSING

11.1 Rationale

WhatsApp usage patterns show that voice messages are frequently preferred over typed text, particularly among users with limited typing comfort. Without audio processing, the system silently ignores every voice note — creating a broken user experience. This enhancement routes audio messages through OpenAI Whisper to produce a text transcript, which is then processed by the existing NLP and FAQ pipeline with no downstream changes required.

11.2 Full audio processing pipeline

```
User sends voice note (WhatsApp .ogg file)
| Webhook receives: message.type = 'audio'
▼ GET /v20.0/{media_id} → obtain download URL
▼ Fetch audio bytes → write to /tmp/audio_{ts}.ogg
▼ POST to OpenAI Whisper /v1/audio/transcriptions
▼ Receive { text: 'transcribed message content' }
▼ fs.unlinkSync(tempPath) → temp file deleted
▼ Inject transcript into existing NLP pipeline
▼ FAQ Engine + Response sent back to user as text
```

11.3 Whisper API specifications

Parameter	Value
Endpoint	POST https://api.openai.com/v1/audio/transcriptions
Model	whisper-1
Input format	multipart/form-data with file field
Accepted formats	.ogg, .mp3, .mp4, .wav, .webm (WhatsApp .ogg files accepted directly)
Max file size	25 MB (typical WhatsApp voice note: 0.1–2 MB)
Language handling	Set language:'fr' for French or omit for auto-detection
Cost	\$0.006 per minute — 30-second note = \$0.003
Latency	1–3 seconds typical response time
API key	Same OPENAI_API_KEY used by GPT-4o-mini — no new credential needed

11.4 Implementation — audio.service.js (complete)

```
const fs = require('fs');
const fetch = require('node-fetch');
const OpenAI = require('openai');

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

async function transcribeAudio(mediaId, accessToken) {
  try {
    // 1. Get media download URL from Meta
    const metaRes = await fetch(
      `https://graph.facebook.com/v20.0/${mediaId}`,
      { headers: { Authorization: `Bearer ${accessToken}` } }
    );
    if (!metaRes.ok) throw new Error('Failed to get media URL from Meta');
    const { url } = await metaRes.json();

    // 2. Download the audio file (.ogg)
    const audioRes = await fetch(url, {
      headers: { Authorization: `Bearer ${accessToken}` }
    });
    if (!audioRes.ok) throw new Error('Failed to download audio file');
    const buffer = await audioRes.buffer();
    const tempPath = `/tmp/audio_${Date.now()}.ogg`;
    fs.writeFileSync(tempPath, buffer);

    // 3. Transcribe with Whisper
    const transcription = await openai.audio.transcriptions.create({
      file: fs.createReadStream(tempPath),
```

```

    model: 'whisper-1',
    // language: 'fr', // uncomment to force French
  });

  fs.unlinkSync(tempPath); // Delete temp file immediately
  return transcription.text;

} catch (error) {
  logger.error('Audio transcription failed:', error);
  return null; // Caller will send fallback reply to user
}
}

module.exports = { transcribeAudio };

```

11.5 Acceptance criteria — Audio Processing

#	Criterion	Method	Threshold
1	Voice note intent correctly detected (FR + EN)	Send 20 test voice notes, review detected intent	Correct in > 80% of cases
2	End-to-end latency (download + transcribe + reply)	Measure total time for 10 voice notes	< 7 seconds at P95
3	Temp file cleanup after each transcription	Check /tmp directory after 10 audio messages	Zero .ogg files remaining
4	Graceful error handling if Whisper API fails	Simulate API timeout	User receives polite fallback text, no crash
5	msg_type stored as 'audio' in messages table	Query DB after 5 voice note tests	All 5 rows show msg_type='audio'

12. ENHANCEMENT 2 — BASIC ANALYTICS DASHBOARD

12.1 Rationale

Without observability, the system is a black box. The analytics module surfaces key operational metrics so the ISETAG team can answer: Is the bot working? Which FAQs are triggered most? What questions does the bot fail to answer? Is the escalation rate improving over time? All the raw data already exists in the database (logged since Sprint 2) — this enhancement is purely an aggregation and visualization layer on top of it.

NOTE No new data collection is required. All analytics are derived from data already stored in the conversations, messages, and faqs tables. This adds zero write load to the database.

12.2 Complete metrics specification

Metric	Description	Source	Visualization
Messages per day	Total incoming messages by date (last 30 days)	messages.sent_at	Bar chart
Top 10 FAQs triggered	Most matched FAQ entries ranked by frequency	messages.faq_matched_id	Horizontal bar
Bot vs Human rate	% conversations handled fully by bot vs escalated	conversations.status	Donut chart
Unmatched questions	Messages where bot had no answer (knowledge gaps)	messages — confidence < 0.5	Ranked table
Channel breakdown	Volume split: WhatsApp / Facebook / Instagram	conversations.channel	Pie chart
Language split	FR vs EN message breakdown	conversations.lang_detected	Donut chart
Avg confidence score	Mean FAQ match confidence over time (daily avg)	messages.confidence	Line chart
Audio vs text ratio	% of messages that were originally voice notes	messages.msg_type	Stat card

12.3 SQL queries for each metric

Messages per day

```
SELECT DATE(sent_at) as day, COUNT(*) as count
FROM messages WHERE direction = 'in'
    AND sent_at >= NOW() - INTERVAL '30 days'
GROUP BY DATE(sent_at) ORDER BY day ASC;
```

Top 10 triggered FAQs

```
SELECT f.id, f.question, f.category, COUNT(m.id) as trigger_count
FROM messages m JOIN faqs f ON m.faq_matched_id = f.id
WHERE m.faq_matched_id IS NOT NULL
GROUP BY f.id, f.question, f.category
ORDER BY trigger_count DESC LIMIT 10;
```

Escalation rate

```
SELECT
    COUNT(*) FILTER (WHERE status = 'bot') AS bot_handled,
    COUNT(*) FILTER (WHERE status = 'escalated') AS escalated,
    ROUND(COUNT(*) FILTER (WHERE status='escalated')*100.0/NULLIF(COUNT(*),0),1) AS pct
FROM conversations WHERE created_at >= NOW() - INTERVAL '30 days';
```

Top 20 knowledge gaps (unmatched messages)

```
SELECT content, COUNT(*) as frequency
FROM messages
WHERE faq_matched_id IS NULL AND direction = 'in'
    AND sent_at >= NOW() - INTERVAL '7 days'
GROUP BY content ORDER BY frequency DESC LIMIT 20;
```

Audio vs text message ratio

```
SELECT
    COUNT(*) FILTER (WHERE msg_type = 'text') AS text_count,
    COUNT(*) FILTER (WHERE msg_type = 'audio') AS audio_count,
    ROUND(COUNT(*) FILTER (WHERE msg_type='audio')*100.0/NULLIF(COUNT(*),0),1) AS
audio_pct
FROM messages WHERE direction = 'in'
    AND sent_at >= NOW() - INTERVAL '30 days';
```

12.4 Analytics acceptance criteria

#	Criterion	Method	Threshold
1	All 6 API endpoints return correct data	Compare API response vs direct psql query results	100% match
2	Analytics page load time	Browser DevTools — network panel timing	Full render < 3 seconds
3	Charts update after new conversations	Add 10 test conversations, refresh page	New data visible
4	Unmatched table reflects real bot failures	Insert 5 known unanswerable questions	All 5 appear in table within 1 minute
5	Audio ratio stat card shows correct %	Send 5 voice notes out of 10 total messages	Card shows 50%

13. COMPLETE ACCEPTANCE CRITERIA

The full system (core + both enhancements) is considered deliverable when all the following criteria pass during final acceptance testing:

#	Criterion	Verification Method	Threshold
1	WhatsApp bot response time (text)	Stopwatch on 50 text messages	< 4 seconds at P95
2	WhatsApp bot response time (audio)	Stopwatch on 20 voice notes	< 7 seconds at P95
3	FAQ matching accuracy	Annotated test set of 50 questions	> 80% correct responses
4	EN/FR language detection	20 messages per language	> 95% correctly detected
5	Voice note intent detection	20 test voice notes in FR + EN	> 80% correct intent
6	Human escalation triggered	3-ambiguous-message scenario	100% of trigger cases covered
7	Conversation summary generated	Review summary after escalation	Readable and relevant content
8	Facebook DM auto-reply	Send DM from test account	Reply received in < 5 seconds
9	FAQ CRUD from Dashboard	Create, edit, deactivate a FAQ	Change reflected immediately on bot
10	PDF upload and WhatsApp sharing	Upload from dashboard + test receipt	PDF received in < 10 seconds
11	Analytics page accuracy	API response vs direct SQL query	100% data match
12	Analytics page load time	Browser DevTools timing	Full render in < 3 seconds
13	Audio temp file cleanup	Check /tmp after 10 voice notes	Zero .ogg files remaining
14	Forged webhook rejected	Send request without valid signature	HTTP 401 returned
15	Production uptime	UptimeRobot monitoring over 7 days	>= 95%

14. TECHNICAL RISK MANAGEMENT

Risk	Likelihood	Impact	Technical Mitigation
Meta App Review delayed or rejected	High	High	Build and test fully in sandbox from Sprint 1; prepare thorough Meta App submission early
WhatsApp monthly quota (1,000 free) exceeded	Medium	Medium	Counter in Redis with alert at 800/1,000; budget for API billing in Month 2+
OpenAI GPT latency > 4 seconds	Low	High	Pass 1 (local) covers ~85% of cases. 3s timeout on GPT call with generic fallback response
Whisper API unavailable or slow	Low	Medium	Fallback: reply 'Could not process your voice note, please type your question'
Inaccurate language auto-detection	Medium	Medium	Prompt user to select language at session start if auto-detect confidence < 0.8
Server instability due to local hosting	High	High	Deploy exclusively on international cloud (Railway/Render) — never on local Cameroonian server
Poorly structured FAQ entries by staff	Medium	Medium	Dashboard validation: required fields, min 3 keywords, response preview before activation
Analytics queries degrading DB performance	Low	Low	Read-only queries; add EXPLAIN ANALYZE before deploying; add indexes on sent_at and faq_matched_id

15. TECHNICAL GLOSSARY

Term	Definition
Webhook	HTTP endpoint on the server that receives real-time event notifications from Meta whenever a message arrives.
HMAC-SHA256	Cryptographic signature algorithm used by Meta to prove the authenticity and integrity of all webhook requests.
NLP	Natural Language Processing — automated analysis of natural language text to extract intent and meaning.
Intent	The underlying purpose of a message (e.g., user asks about tuition fees → intent = 'fees').
Cosine Similarity	Mathematical measure of similarity between two text vectors, used in FAQ Pass 1 to rank matches.
TTL	Time To Live — the lifespan of a Redis entry before it is automatically deleted.
JWT	JSON Web Token — a compact, signed token used to authenticate admin dashboard sessions.
Connection Pool	A group of pre-opened database connections reused across requests to avoid per-request TCP overhead.
Rate Limiting	Server-side mechanism limiting how many requests a user can make in a defined time window.
CDN	Content Delivery Network — geographically distributed file servers for fast delivery of static assets (PDFs, images).
Escalation	The process of transferring a conversation from the automated bot to a human agent.
Whisper	OpenAI's open-source Speech-to-Text model, accessed here via API, that converts audio files to text transcripts.
Parameterized Query	SQL technique that separates the query structure from user-supplied values, preventing SQL injection.
Pass 1 / Pass 2	Two-stage FAQ matching: Pass 1 uses local keyword matching (fast, free); Pass 2 uses GPT (slower, costs API tokens).
Transcript	The text output produced by the Whisper API after processing a voice message audio file.
Knowledge Gap	A user question that the bot failed to match to any FAQ, recorded in the analytics unmatched table.

End of Complete Technical Specifications Document — TSD v1.1

ISETAG University — WhatsApp FAQ Bot & Social Media AutoResponder

Architecture | API Integrations | Sprint Plan | Audio Processing | Analytics | July 2025